# Lenguaje SQL

Descargar estos apuntes

## Índice


- Lenguaje SQL
  - Lenguaje de Manipulación de Datos
  - Consultas `SELECT` básicas
    - Consultas con operaciones numéricas y literales
    - Consultas con funciones predefinidas de información
    - Consultas básicas a tablas
      - Ejemplos de las primeras consultas
    - Consultas con selección de registros
    - Consultas con varias condiciones
    - Consultas con funciones
  - Consultas `SELECT` de AGRUPACIÓN
    - Consultas de agrupación
    - Ejemplos de las primeras consultas agrupadas
  - Consultas SELECT multitabla
    - Consultas multitabla de cruce (relación 1-N)
    - Consultas multitabla mediante INNER JOIN
    - La cláusula LEFT JOIN y RIGHT JOIN
    - Consultas multitabla mediante operaciones conjuntistas
      - La cláusula `UNION`
      - La cláusula `INTERSECT`
      - La cláusula `EXCEPT`
    - Consultas multitabla mediante SUBCONSULTAS
      - Subconsultas en `WHERE` con operador de comparación
      - Subconsultas en `WHERE` con operador `IN`
      - Subconsultas en `WHERE` con operador `EXISTS`
      - Subconsultas en `FROM`
  - Transacciones y bloqueos
    - Transacciones
      - Transacciones en MariaDB
        - Autocommit
      - Transacciones concurrentes
      - Pérdida de actualizaciones (Lost Updated)
      - Problema de lectura sucia (Dirty Read)
      - Problema lectura no repetible (Non-Repeatable Read)
      - Problema de lectura fantasma (Phantom Read)

Propiedades ACID de la transacción ideal
Niveles de aislamiento
▼ Bloqueos
Casos de uso de bloqueos compartidos
Casos de uso de bloqueos exclusivos
▼ Niveles de bloqueo
Bloqueo exclusivo a nivel de fila
Bloqueo compartido a nivel de fila
Bloqueo compartido a nivel de tabla
▼ Usuarios y privilegios
Creación de usuarios SQL
Asignación de privilegios
Eliminar permisos
Creación de roles
Optimización de consultas

# Lenguaje de Manipulación de Datos

El **Lenguaje de Manipulación de Datos** (**LMD**, en inglés Data Manipulation Language, **DML**) es un lenguaje proporcionado por el sistema de gestión de base de datos que permite a los usuarios de la misma llevar a cabo las tareas de consulta o manipulación de los datos (inserción, borrado, actualización y consultas) basado en el modelo de datos adecuado.

> **Referencias**
>
> MySQL Oficial – Sintaxis de las instrucciones DML

Las **consultas** recuperan información de las tablas de una base de datos mediante la selección de campos, la realización de filtros y la transformación de los datos recuperados.

El uso de consultas permite:

- **Elegir tablas.** Se puede obtener información de una sola tabla o de varias.
- **Elegir campos.** Se pueden especificar los campos a visualizar de cada tabla.
- **Elegir registros.** Se pueden seleccionar los registros a mostrar en la hoja de respuestas dinámica, especificando un criterio.
- **Ordenar registros.** Se puede ordenar la información en forma ascendente o descendente.
- **Realizar cálculos.** Se pueden emplear las consultas para hacer cálculos con los datos de las tablas.

Además, en consultas más complejas se puede:

- **Crear tablas.** Se puede generar otra tabla a partir de los datos combinados de una consulta. La tabla se genera a partir de la hoja de respuestas dinámica.

- **Consultas encadenadas.** Utilizar una consulta como origen de datos para otras consultas (subconsulta). Se pueden crear consultas adicionales basadas en un conjunto de registros seleccionados por una consulta anterior.

# Consultas `SELECT` básicas

## Consultas con operaciones numéricas y literales

Comenzaremos por utilizar la instrucción `SELECT` como si fuera una calculadora. Para ello utilizaremos los siguientes operadores aritméticos:

**Operadores aritméticos**

| Operador | Función |
|---|---|
| + | Suma |
| - | Resta |
| * | Producto o Multiplicación |
| / | División |
| DIV | División entera |
| MOD o % | Resto de la división |

Los siguientes ejemplos evaluan las expresiones que hay después de la cláusula `SELECT` y muestran el resultado por pantalla.

### Ejemplo 1

La última sentencia muestra tres columnas.

```
-- Salida: 3000
SELECT 20 * 150;

-- Salida: 14.5
SELECT ((5 * 4.5) / 3) + 7;

-- Salida: 33.333333333, 33, 1
SELECT 100 / 3, 100 DIV 3, 100 MOD 3;
```

### Ejemplo 2

Para el uso de literales o cadenas de caracteres basta con colocar unas comillas.

```
-- Salida: Felicidades
SELECT 'Felicidades';

-- Salida: Precio, 25 * 1.21 =, 30.25
SELECT 'Precio', '25 * 1.21 =', 25 * 1.21;
```

**Referencias**

MySQL – Operadores aritméticos
MySQL Oficial - Operadores

## Consultas con funciones predefinidas de información

A través de funciones ya predefinidas en el MySQL podemos obtener información sobre diferentes aspectos del estado de nuestra conexión y del sistema. Algunos ejemplos son:

**Ejemplo 1**

La **versión** de MySQL y nuestro identificador (**ID**) de conexión.

```
SELECT version(), connection_id();
```

**Ejemplo 2**

El usuario actual con el que estamos conectados y la base de datos seleccionada.

```
SELECT user(), database();
```

**Ejemplo 3**

Fecha y hora ( `now()` o sólo la fecha `current_date()` ) del sistema.

```
SELECT now(), current_date();
```

**Referencias**

MySQL – Funciones de información
MySQL Oficial – Funciones de información

# Consultas básicas a tablas

```
SELECT [DISTINCT] campos
[FROM table_references]
[WHERE where_definition]
[ORDER BY {col_name | expr | position} [ASC | DESC] , ...]
[LIMIT [offset,] row_count]
```

✋ **Importante**

El orden de las cláusulas, **obligatoriamente**, es:
**SELECT … FROM … WHERE … ORDER … LIMIT**

## Ejemplos de las primeras consultas

Puedes probar estas consultas sobre la base de datos jardinería.

### Ejemplo 1

**Tablas completas**

Mostrar todos los campos de todos los clientes.

```
SELECT *
FROM clientes;
```

### Ejemplo 2

**Seleccionar campos a mostrar**

Mostrar los campos `CodigoCliente`, `NombreCliente`, `Telefono`, `Ciudad`, `Region`, `Pais` de todos los clientes.

```
SELECT CodigoCliente, NombreCliente, Telefono, Ciudad, Region, Pais
FROM clientes;
```

### Ejemplo 3

**Campos calculados**

Mostrar los campos `CodigoCliente`, `NombreCliente`, `LimiteCredito` de todos los clientes añadiendo un campo calculado que es el `LimiteCreditoMensual` como `LimiteCredito / 12`. Haced la division entera para evitar decimales. Utilizaremos el **alias de campo** con la palabra reservada `AS`.

```
SELECT CodigoCliente, NombreCliente, LimiteCredito, LimiteCredito DIV 12 AS LimiteCreditoMensual
FROM clientes;
```

### Ejemplo 4

**No mostrar repetidos**

Mostrar las regiones (campo `Region`) todos los clientes evitando resultandos repetidos.

```
SELECT DISTINCT Region
FROM clientes;
```

### Ejemplo 5

**Ordenar registros**

Mostrar todos los campos de todos los clientes ordenados por `LimiteCredito` ordenado descendentemente.

```
SELECT * FROM clientes
ORDER BY LimiteCredito DESC;
```

### Ejemplo 6

**Limitar el número de registros a mostrar del resultado**

Utilizando la consulta del ejemplo anterior mostrar:

- Sólo los 5 primeros registros

```
SELECT *
FROM clientes
ORDER BY LimiteCredito DESC
LIMIT 5;
```

- Empezando en el tercer registro, mostrar 10 registros

```
SELECT *
FROM clientes
ORDER BY LimiteCredito DESC
LIMIT 2,10;
```

## Consultas con selección de registros

Para poder seleccionar registros es necesario indicar la condición que deben cumplir los campos de un registro para ser mostrado. Para ello se utiliza la sección `WHERE` de la instrucción SQL.
Los operadores relacionales que nos permiten comparar el valor de los campos son:

**Operadores relacionales**

| Operador | Función |
|---|---|
| = | Igual a |
| < | Menor que |
| > | Mayor que |
| <= | Menor o igual |
| >= | Mayor o igual |
| LIKE | Patrón que debe cumprir un campo cadena |
| BETWEEN | Intervalo de valores |
| IN | Conjunto de valores |
| IS NULL | Es nulo |
| IS NOT NULL | No es nulo |

Veamos unos cuantos ejemplos:

### Ejemplo 1

**Buscar un registro por el valor de su clave**

Mostrar todos los campos del cliente con código igual a 12.

```
SELECT *
FROM clientes
WHERE CodigoCliente = 12;
```

> 📌 Al ser el campo clave, el resultado sólo mostrará un registro.

### Ejemplo 2

**Buscar registros por el valor de un campo**

Mostrar todos los campos de los clientes de la `Región` *Madrid*.

```
SELECT *
FROM clientes
WHERE Region = 'Madrid';
```

> 📌 Al NO ser un campo clave, el resultado puede mostrar más de un registro.

### Ejemplo 3

**Buscar registros por comparación del valor de un campo**

Mostrar todos los campos de los clientes con Límite de Crédito mayor de 50000€.

```sql
SELECT *
FROM clientes
WHERE LimiteCredito > 50000;
```

### Ejemplo 4

**Buscar registros por comparación del valor de un campo**

Mostrar todos los campos de los clientes con Límite de Crédito entre 10000€ y 60000€.

```sql
SELECT *
FROM clientes
WHERE LimiteCredito BETWEEN 10000 AND 60000;
```

### Ejemplo 5

**Buscar registros por comparación del valor de un campo**

Mostrar todos los campos de los clientes con la `Región` nula.

```sql
SELECT *
FROM clientes
WHERE Region IS NULL;
```

### Ejemplo 6

**Ejemplo combinando con el ejemplo anterior**

Mostrar los Países de los clientes con la `Región` nula, sin repetir valores.

```sql
SELECT DISTINCT Pais
FROM clientes
WHERE Region IS NULL;
```

### Ejemplo 7

**Ejemplo con el operador `LIKE`**

Mostrar los Empleados cuyo `email` contenga *jardin*.

```sql
SELECT *
FROM Empleados
WHERE Email LIKE '%jardin%';
```

📌 **%** es una cadena de caracteres cualquiera y **_** es un sólo carácter cualquiera.

### Ejemplo 8

**Ejemplo con el operador `IN`**

Mostrar los productos de las gamas *Aromáticas*, *Herramientas* y *Ornamentales*.

```
SELECT *
FROM productos
WHERE Gama IN ('Aromáticas', 'Herramientas', 'Ornamentales');
```

### Referencias

MySQL – Consultas
MySQL – Operadores de comparación
MySQL Oficial – Operadores de comparación

## Consultas con varias condiciones

Para poder realizar varias varias condiciones necesitamos combinarlas con operadores lógicos, que en MySQL son:

**Operadores lógicos**

| Operador | Función |
|---|---|
| AND | Y |
| OR | O |
| NOT | No |

Veamos unos cuantos ejemplos:

### Ejemplo 1

**Ejemplo con varias condiciones**

Mostrar codigo ( `CodigoProducto` ) y `nombre` de los Productos que sean de la `Gama` *Frutales* y `CantidadEnSock` sea mayor que 50 unidades. Mostremos también los campos implicados en las condiciones para comprobar el resultado.

```
SELECT CodigoProducto, Nombre, Gama,
FROM productos
WHERE Gama = 'Frutales' AND CantidadEnStock > 50;
```

### Ejemplo 2

**Ejemplo con varias condiciones**

Mostrar código y nombre de los Clientes que sean de la `Ciudad` *Madrid* o *Barcelona*. Mostremos también los campos implicados en las condiciones para comprobar el resultado.

```
SELECT CodigoCliente, NombreCliente, Ciudad
FROM clientes
WHERE Ciudad = 'Madrid' OR Ciudad = 'Barcelona';
```

### Referencias

MySQL – Consultas y operadores de comparación (curso)
MySQL Oficial – Operadores lógicos

## Consultas con funciones

Las funciones nos permiten realizar transformaciones de los datos para obtener información. Existen multitud de funciones que procesan diferentes tipos de datos. Las funciones predefinidas en MySQL se pueden utilizar tanto en campos calculados como en condiciones `WHERE`.A continuación se muestran algunas de las funciones más utilizadas agrupadas por tipos de datos. Las funciones se tratarán en mayor profundidad en la siguiente unidad.

**Funciones de cadenas de caracteres**

| Función | Descripción |
|---|---|
| CONCAT(cad1, cad2, ...) | Concatena cadenas |
| UPPER(cad), UCASE(cad) | Pasa a mayúsculas una cadena |
| LOWER(cad), LCASE(cad) | Pasa a minúsculas una cadena |
| LTRIM(cad) | Elimina de la cadena los espacios iniciales y finales |
| LEFT(cad, X) | Obtiene los X primeros caracteres de la cadena |
| RIGHT(cad, X) | Obtiene los X últimos caracteres de la cadena |
| LENGTH(cad) | Longitud de una cadena |
| REPLACE(cad, ant, pos) | Obtiene una cadena tomando *cad* como origen y cambiando *ant* por *pos* |

**Funciones numéricas de un campo**

| Función | Descripción |
|---|---|
| RAND() | Número aleatorio decimal entre 0 y 1 (menor que 1) |
| POW(num, exp) | Obtiene la potencia *num* elevado a *exp* |
| FLOOR(num) | Obtiene la parte entera de un número decimal |
| ROUND(num, X) | Redondea un número a X decimales |
| SIGN(num) | Devuelve 1 para positivo, 0 para 0 y -1 para negativo |
| ABS(num) | Obtiene el valor absoluto de un número |

**Funciones de fechas de un campo**

| Función | Descripción |
|---|---|
| YEAR(campo) | Muestra el año del valor de un campo de tipo fecha |
| MONTH(campo) | Muestra el mes del valor de un campo de tipo fecha |
| DAY(campo) | Muestra el día del valor de un campo de tipo fecha |
| DAYNAME(campo) | Muestra el nombre del día de la semana del campo |
| DAYOFWEEK(campo) | Devuelve un número indicando el día de la semana de un campo fecha: *1=Domingo, 2=Lunes, 3=Martes, 4=Miércoles, 5=Jueves, 6=Viernes, 7=Sábado.* |

**Funciones numéricas con varios registros**

| Función | Descripción |
|---|---|
| COUNT(*) | Cuenta los registros seleccionados |
| MIN(campo) | Valor mínimo del campo de los registros seleccionados |
| MAX(campo) | Valor máximo del campo de los registros seleccionados |
| SUM(campo) | Suma de los valores del campo |
| AVG(campo) | Media de los valores del campo |

**Referencias**

MySQL w3schools – Funciones de MySQL
MySQL Oficial – Funciones

***Funciones para campos calculados y para condiciones***

**Ejemplo 1**

**Ejemplo con funciones**

Mostrar código ( `CodigoProducto` ), `nombre` y `precio de venta al público` de los productos, pero ese precio debe ser con IVA incluido, es decir, agregarle al `PrecioVenta` el 21% multiplicándolo por 1.21. Asignar el alias `pvp` al nuevo campo calculado.

```
SELECT CodigoProducto, Nombre, PrecioVenta, ROUND(PrecioVenta * 1.21, 2) AS pvp
FROM Productos;
```

**Ejemplo 2**

**Ejemplo con funciones en las condiciones**

Seleccionar en la consulta anterior los productos que tengan el nuevo campo `pvp` mayor de 100€.

```sql
SELECT CodigoProducto, Nombre, PrecioVenta, ROUND(PrecioVenta * 1.21, 2) AS pvp
FROM Productos
WHERE ROUND(PrecioVenta * 1.21, 2) > 100;
```

***Funciones para manejo de cadenas de caracteres***

### Ejemplo 3

**Ejemplo con funciones**

Seleccionar de los empleados el nombre completo ( `NombreCompleto` ) concatenando el nombre y los dos apellidos.

```sql
SELECT CONCAT(Nombre, ' ', Apellido1, ' ', Apellido2) AS NombreCompleto
FROM empleados;
```

### Ejemplo 4

**Ejemplo con funciones**

Obtener la inicial del `nombre` de todos los empleados.

```sql
SELECT LEFT(Nombre, 1) AS inicial
FROM empleados;
```

### Ejemplo 5

**Ejemplo con funciones**

Obtener el `nombre` de los empleados todo en mayúsculas.

```sql
SELECT UCASE(Nombre)
FROM empleados;
```

El mismo resultado alternativamente se puede obtener con:

```sql
SELECT UPPER(Nombre)
FROM empleados;
```

### Ejemplo 6

**Ejemplo con funciones**

Obtener el código de las oficinas ( `CodigoOficina` ) pero sustituyendo el guión por la barra, es decir, el '-' por '/'.

```
SELECT REPLACE(CodigoOficina, '-', '/')
FROM oficinas;
```

En una misma consulta se pueden utilizar varias funciones combinadas.

### Ejemplo 7

**Ejemplo con funciones**

Obtener el email de cada empleado teniendo en cuenta que el usuario es su nombre en minúsculas y el dominio *@iesdoctorbalmis.com*.

```
SELECT CONCAT(LCASE(nombre), '@iesdoctorbalmis.com') AS email
FROM empleados;
```

### Ejemplo 8

**Ejemplo con funciones**

Obtener la abreviatura del nombre y primer apellido de los empleados concatenando la inicial del nombre y la inicial del apellido1.

```
SELECT CONCAT(LEFT(Nombre, 1), LEFT(Apellido1, 1)) AS inicial
FROM empleados;
```

***Funciones numéricas***

### Ejemplo 9

**Ejemplo con funciones**

Obtener un número aleatorio entre 0 y 9.

```
SELECT FLOOR(10 * RAND());
```

***Funciones de fechas***

### Ejemplo 10

**Ejemplo con funciones**

Mostrar en campos diferentes el año, el mes y el día de la fecha de los pedidos.

```
SELECT YEAR(FechaPedido), MONTH(FechaPedido), DAY(FechaPedido)
FROM pedidos;
```

### Ejemplo 11

**Ejemplo con funciones**

Mostrar todos los campos de pedidos realizados en enero del 2009.

```
SELECT *
FROM pedidos
WHERE YEAR(FechaPedido) = 2009 AND MONTH(FechaPedido) = 1;
```

### Ejemplo 12

**Ejemplo con funciones**

Mostrar todos los campos de pedidos realizados en lunes.

```
SELECT *
FROM pedidos
WHERE DAYOFWEEK(FechaEntrega) = 2;
```

***Funciones numéricas con varios registros***

Estás funciones nos permiten realizar cálculos sobre varios registros seleccionados. Se utilizan en consultas de agrupación que veremos más adelante, pero también se pueden utilizar en consultas sencillas. En este caso, la función se aplica a todos los registros seleccionados por la consulta.

### Ejemplo 13

**Ejemplo con funciones**

Mostrar el número total de pagos que se han recibido.

```
SELECT COUNT(*)
FROM pagos;
```

### Ejemplo 14

**Ejemplo con funciones**

Mostrar la cantidad en euros total que nos han ingresado.

```
SELECT SUM(Cantidad)
FROM pagos;
```

### Ejemplo 15

**Ejemplo con funciones**

Mostrar el importe del pago más pequeño y el importe de pago más grande recibido.

```
SELECT MIN(Cantidad), MAX(Cantidad)
FROM pagos;
```

# Consultas `SELECT` de AGRUPACIÓN

## Consultas de agrupación

Cuando necesitamos agrupar varios registros para realizar operaciones para sumar, contar, o calcular la media, el mínimo o el máximo, necesitaremos realizar una instrucción `SELECT` indicando qué registros agrupamos, es decir, qué campos mostramos de los registros comunes y sobre qué campos realizamos la operación.
La sintaxis para realizar una consulta agrupada a una tabla es:

```
SELECT [DISTINCT] campos
[FROM table_references]
[WHERE where_definition]
[GROUP BY {col_name | expr | position} [ASC | DESC], ... ]
[HAVING where_definition]
[ORDER BY {col_name | expr | position} [ASC | DESC] , ...]
[LIMIT [offset,] row_count]
```

✋ **Importante**

El orden de las cláusulas, **obligatoriamente**, es:
**SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER … LIMIT**

## Ejemplos de las primeras consultas agrupadas

Con XAMPP-MySQL importar la **BD de jardinería** que facilitará el profesor.

### Ejemplo 1

**Contar**

Mostrar el número de clientes (nombrar el nuevo campo `NumClientes`) que tenemos en cada `ciudad`.

```
SELECT Ciudad, COUNT(*) AS NumClientes
FROM clientes
GROUP BY Ciudad;
```

### Ejemplo 2

**Sumar operaciones numéricas**

Mostrar el pedido y el importe total (`ImpTotal`) de todas las líneas de cada pedido. En la tabla detallepedidos tenemos el `CodigoPedido`, la `Cantidad` y el `PrecioUnidad` de cada artículo del pedido. Deberemos sumar `Cantidad * PrecioUnidad` de cada línea y luego sumarlas todas.

```
-- Sin Agrupar
SELECT CodigoPedido, Cantidad * PrecioUnidad AS ImpLinea
FROM detallepedidos;

-- Agrupando por Pedido
SELECT CodigoPedido, SUM(Cantidad * PrecioUnidad) AS ImpTotal
FROM detallepedidos
GROUP BY CodigoPedido;
```

Podemos añadir también filtros que deban cumplir los registros mediante condiciones en la cláusula `WHERE`.

### Ejemplo 3

**Agrupaciones con condiciones `WHERE`**

Mostrar el número de artículos (`NumArticulos`) de cada Gama cuyas `PrecioVenta` mayor que 20€.

```
SELECT Gama, COUNT(*) AS NumArticulos
FROM productos
WHERE PrecioVenta > 20
GROUP BY Gama;
```

Pero puede ser que la condición a cumplir sea sobre los campos calculados. En estos casos, la condición irá en la cláusula `HAVING`.

### Ejemplo 4

**Agrupaciones con condiciones `HAVING`**

Mostrar las gamas (`Gama`) de artículos que tengan más de 100 diferentes. Mostrar también el número de

artículos ( `NumArticulos` ).

```
SELECT Gama, COUNT(*) AS NumArticulos
FROM productos
GROUP BY Gama
HAVING COUNT(*) > 100;
```

Incluso podemos tener consultas que combinen `WHERE` y `HAVING` simultáneamente.

### Ejemplo 5

**Agrupaciones con condiciones `WHERE` y `HAVING`**

Mostrar los clientes que hayan realizado más de 1 pago de cantidad superior a los 3000€.

```
SELECT CodigoCLiente, COUNT(*) AS NumPagos
FROM pagos
WHERE Cantidad > 3000
GROUP BY CodigoCliente
HAVING COUNT(*) > 1;
```

# Consultas SELECT multitabla

## Consultas multitabla de cruce (relación 1-N)

Las consultas multitabla nos van a permitir procesar información de varias tablas conjuntamete. La sintaxis es la misma que hemos visto anteriormente para la sintaxis `SELECT` , pero en la cláusula `FROM` tendremos **varias tablas**.

### Ejemplo 1

**SELECT multitabla**

Mostrar los valores de la tabla **pagos** añadiendo el campo `NombreCliente` .

```
SELECT clientes.NombreCliente, pagos.*
FROM pagos, clientes
WHERE pagos.CodigoCliente = clientes.CodigoCliente;
```

- ¿Cuántos registros tienen la tabla pagos?
- ¿Cuántos registros tienen la tabla clientes?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición `WHERE` , ¿cuántos registros obtenemos? Justifícalo.

En este primer ejemplo se puede observar tanto en la cláusula `SELECT` como en la `WHERE` que para referirse a un campo de una tabla utilizamos la sintaxis: ***tabla.campo***.
Por economía del lengaje, podemos definir alias de tablas y utilizar estos para referirse a los campos como: ***alias.campo***.
En una consulta multitabla no es necesario utilizar el prefijo de tabla (o alias) para referirse a un campo siempre y cuando no exista un campo con el mismo nombre en más de una de las tablas seleccionadas.
En las consultas sobre una tabla no es necesario ni utilizar el nombre de la tabla ni un alias de esta para referisrse a los campos de la tabla. No obstante, en muchos entornos de trabajo es ventajoso utilizar el nombre de tabla o alias ya que, en este caso, nos ofrece la opción de autocompletar y selección de campos. *PhpMyAdmin* nos brinda esa posibilidad.

A continuación veamos una variante del primer ejemplo con el fin de aclarar estos conceptos.

### Ejemplo 1b

**SELECT multitabla**
Mostrar los valores de la tabla **pagos** añadiendo el campo `NombreCliente`. Utilizar alias a tablas y no utilizar prefijo de alias si no es necesario.

```
SELECT NombreCliente, p.*
FROM pagos p, clientes c
WHERE p.CodigoCliente = c.CodigoCliente;
```

Hemos utilizado **p** como alias de la tabla `pagos` y **c** como alias de la tabla `cliente`. Ahora accedemos a los campos de la tabla mediante el alias y no con el nombre de la tabla. La consulta se observa que:

- Con **p.*** seleccionamos todos los campos de la tabla pagos.
- En la condición **es necesario utilizar el alias** puesto que tanto en la tabla pagos como clientes hay un campo que se llama `CodigoCliente`.
- No hace falta utilizar un alias para acceder al campo `NombreCliente` puesto que sólo exsite en la tabla clientes.

### Ejemplo 2

**SELECT multitabla**
Mostrar los valores de la tabla **pedidos** añadiendo el campo `NombreCliente`.

```
SELECT clientes.NombreCliente, pedidos.*
FROM pedidos, clientes
WHERE pedidos.CodigoCliente = clientes.CodigoCliente;
```

- ¿Cuántos registros tienen la tabla pedidos?
- ¿Cuántos registros tienen la tabla clientes?
- ¿Cuántos registros devuelve la consulta anterior?

- Si se elimina la condición `WHERE`, ¿cuántos registros obtenemos? Justifícalo.

### Ejemplo 3

**SELECT multitabla**

Mostrar los valores de la tabla **empleados** añadiendo los campos `Ciudad`, `Pais` y la oficina del país `CodigoOficina`.

```
SELECT oficinas.Ciudad, oficinas.Pais, empleados.*
FROM oficinas, empleados
WHERE oficinas.CodigoOficina = empleados.CodigoOficina;
```

- ¿Cuántos registros tienen la tabla oficinas?
- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición `WHERE`, ¿cuántos registros obtenemos? Justifícalo.

En relaciones reflexivas, debemos cruzar una tabla consigo misma. Para poder hacer esto, tenemos que, al menos, asignar un alias a una de las tablas para diferenciarlas. Veamos un ejemplo:

### Ejemplo 4

**SELECT multitabla con relación reflexiva**

Mostrar los valores de la tabla **empleados** añadiendo los campos `Nombre` y `Apellido1` de su jefe.

```
SELECT jefes.Nombre, jefes.Apellido1, empleados.*
FROM empleados, empleados AS jefes
WHERE empleados.CodigoJefe = jefes.CodigoEmpleado;
```

Otra forma de escribir la consulta es:

```
SELECT jefes.Nombre, jefes.Apellido1, trabajadores.*
FROM empleados AS trabajadores, empleados AS jefes
WHERE trabajadores.CodigoJefe = jefes.CodigoEmpleado;
```

- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición `WHERE`, ¿cuántos registros obtenemos? Justifícalo.

### Ejemplo 5

**SELECT multitabla con varias tablas**

Mostrar los siguientes valores:

- De **pedidos**: `CodigoPedido` y `FechaPedido`
- De **detallepedidos**: `CodigoProducto` y `Cantidad`
- De **productos**: `Nombre` y `Gama`

```
SELECT pedidos.CodigoPedido, pedidos.FechaPedido, detallepedidos.CodigoProducto, detallepedidos.Cantidad, productos.N
FROM pedidos, detallepedidos, productos
WHERE pedidos.CodigoPedido = detallepedidos.CodigoPedido AND detallepedidos.CodigoProducto = productos.CodigoProducto
```

- ¿Cuántos registros tienen la tabla empleados?
- ¿Cuántos registros devuelve la consulta anterior?
- Si se elimina la condición `WHERE`, ¿cuántos registros obtenemos? Justifícalo.

Se puede observar que cuando se muestran datos de la tabla padre de relaciones con cardinalidad 1-N, estos se repiten ya que pueden tener muchos hijos. En el ejemplo anterior se puede ver claramente que los datos de la tabla pedidos se repiten tantas veces como líneas de detalle tengan.

## Consultas multitabla mediante INNER JOIN

Las consultas utilizando la cláusula **INNER JOIN** son totalmente equivalentes a las consultas multitabla de cruce y se pueden ejecutar con la cláusula:

```
tabla1 INNER JOIN tabla2 ON condicion
```

En el siguiente ejemplo se muetra la equivalencia entre ambas formas de escribir la consulta multitabla.

**Ejemplo 1**

**SELECT multitabla con `INNER JOIN`**

Mostrar los valores de la tabla **pagos** añadiendo el campo `NombreCliente`.

```
SELECT clientes.NombreCliente, pagos.*
FROM (clientes INNER JOIN pagos ON clientes.CodigoCliente = pagos.CodigoCliente);
```

La consulta multitabla de cruce equivalente es:

```
SELECT clientes.NombreCliente, pagos.*
FROM clientes, pagos
WHERE clientes.CodigoCliente = pagos.CodigoCliente;
```

Veamos otro ejemplo de consulta multitabla:

### Ejemplo 2

**SELECT multitabla con `INNER JOIN`**

Mostrar los valores de la tabla **detallepedidos** pero añadiendo los campos `FechaPedido` y `Estado` de la tabla pedidos. Comprueba que el resultado contiene el mismo número de registros que detallepedidos.

```
SELECT pedidos.FechaPedido, pedidos.Estado, detallepedidos.*
FROM (detallepedidos INNER JOIN pedidos ON detallepedidos.CodigoPedido = pedidos.CodigoPedido);
```

## La cláusula LEFT JOIN y RIGHT JOIN

En las consultas multitabla con **INNER JOIN**, **es posible que haya registros que no se muestren por no haber cruce entre entre ellos**. Por ejemplo, si queremos saber el número de pedidos realizados por cada cliente, podríamos pensar en realizar la siguiente consulta:

### Ejemplo 3

**SELECT multitabla con `INNER JOIN`**

Mostrar el número de pedidos realizados por cada cliente.

```
SELECT clientes.CodigoCliente, clientes.NombreCliente, COUNT(pedidos.CodigoPedido)
FROM clientes INNER JOIN pedidos ON clientes.CodigoCliente = pedidos.CodigoCliente
GROUP BY clientes.CodigoCliente;
```

En la tabla de clientes hay 36 registros, pero el resultado de la consulta sólo muestra 19. Esto ocurre porque hay clientes que no han realizado todavía ningún pedido.

Para evitar esto, se introduce un cambio en la consulta que permite incluir todos los registros de una tabla del join, que en nuestro caso es clientes. Lo haremos con la cláusula `LEFT JOIN` de la relación **clientes-pedidos**:

### Ejemplo 4

**SELECT multitabla con `LEFT JOIN`**

Mostrar el número de pedidos realizados por cada cliente, incluyendo aquellos que no han realizado ninguno.

```
SELECT clientes.CodigoCliente, clientes.NombreCliente, COUNT(pedidos.CodigoPedido)
FROM clientes LEFT JOIN pedidos ON clientes.CodigoCliente = pedidos.CodigoCliente
GROUP BY clientes.CodigoCliente;
```

Hay que destacar que en los clientes que no han realizado ningún pedido, el resultado de la función `COUNT` es 0. Sin embargo, si en vez de `COUNT(pedidos.CodigoPedido)` hubiéramos utilizado `COUNT(*)`, el resultado

hubiera sido 1, ya que el registro del cliente existe.

Veamos otro ejemplo:

**Ejemplo 5**

**SELECT multitabla con `LEFT JOIN`**

Mostrar el `CodigoProducto` y nombre de la tabla **productos** junto con la suma de la cantidad ( `SumCantidad` ) pedida en todos los pedidos existentes.

```
SELECT productos.CodigoProducto, productos.Nombre, SUM(detallepedidos.Cantidad) AS SumCantidad
FROM productos LEFT JOIN detallepedidos ON productos.CodigoProducto = detallepedidos.CodigoProducto
GROUP BY productos.CodigoProducto;
```

La cláusula `RIGHT JOIN` permitiría que se mostrarán los de la segunda tabla en la relación, en vez de `LEFT` que muestra la primera. También se podrían invertir las tablas en la consulta con `LEFT JOIN` para obtener el mismo resultado. El siguiente ejemplo muestra el uso de `RIGHT JOIN` y su equivalencia con `LEFT JOIN`.

**Ejemplo 6**

**SELECT multitabla con `RIGHT JOIN`**

Mostrar el número de pedidos realizados por cada cliente, incluyendo aquellos que no han realizado ninguno.

```
SELECT clientes.CodigoCliente, clientes.NombreCliente, COUNT(pedidos.CodigoPedido)
FROM pedidos RIGHT JOIN clientes ON clientes.CodigoCliente = pedidos.CodigoCliente
GROUP BY clientes.CodigoCliente;
```

Hemos utilizado `RIGHT JOIN` para que se muestren todos los registros de la tabla clientes, que es la segunda tabla en la relación. El resultado es el mismo que el obtenido con `LEFT JOIN` en el ejemplo 4.

# Consultas multitabla mediante operaciones conjuntistas

Para realizar consultas multitabla también podemos utilizar operaciones conjuntistas como `UNION`, `INTERSECT` y `EXCEPT`. Hay que tener en cuenta que estas operaciones conjuntistas no son tan flexibles como las consultas multitabla mediante cruce o joins, ya que requieren que las consultas tengan el mismo número de campos y tipos de datos compatibles.

## La cláusula `UNION`

La cláusula `UNION` nos permite unir el resultado de dos consultas. El resultado de la unión mostrará todos los registros de las dos consultas, pero sin repetir los registros comunes a ambas consultas.

Por ejemplo, tenemos por un lado los clientes que han realizado pagos posteriores al 31/01/2009:

```
SELECT DISTINCT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31';
```

Los clientes que han realizado estos pagos son: 3, 15, 16, 19, 23 y 27.

Por otra parte, tenemos los clientes que han realizado pedidos posteriores al 31/03/2009:

```
SELECT DISTINCT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

Los clientes que han realizado estos pedidos son: 16, 23, 27 y 27.

Si queremos obtener el conjunto de clientes que han realizado pagos posteriores al 31/01/2009 o pedidos posteriores al 31/03/2009, podemos utilizar la cláusula `UNION` para unir ambas consultas:

### Ejemplo 1

**SELECT multitabla con `UNION`**
Mostrar el `CodigoCliente` de los clientes que han realizado pagos posteriores al 31/01/2009 o pedidos posteriores al 31/03/2009.

```
SELECT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31'
UNION
SELECT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

Los Codigos de Cliente retornados por la consulta son: 3, 15, 16, 19, 23, 27 y 30.

## La cláusula `INTERSECT`

La cláusula `INTERSECT` nos permite obtener los registros comunes a dos consultas.

Si queremos obtener el conjunto de clientes que han realizado pagos posteriores al 31/01/2009 y pedidos posteriores al 31/03/2009, podemos utilizar la cláusula `INTERSECT` para cruzar ambas consultas:

### Ejemplo 2

**SELECT multitabla con `INTERSECT`**
Mostrar el `CodigoCliente` de los clientes que han realizado pagos posteriores al 31/01/2009 y pedidos

posteriores al 31/03/2009.

```
SELECT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31'
INTERSECT
SELECT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

El Código de Cliente retornado por la consulta es: 16, 23 y 27.

### La cláusula `EXCEPT`

La cláusula `EXCEPT` nos permite obtener los registros de la primera consulta que no están en la segunda consulta.

Si queremos obtener el conjunto de clientes que han realizado pagos posteriores al 31/01/2009 pero que no han realizado pedidos posteriores al 31/03/2009, podemos utilizar la cláusula `EXCEPT` para restar ambas consultas:

**Ejemplo 3**

**SELECT multitabla con `EXCEPT`**
Mostrar el `CodigoCliente` de los clientes que han realizado pagos posteriores al 31/01/2009 pero que no han realizado pedidos posteriores al 31/03/2009.

```
SELECT CodigoCliente
FROM pagos
WHERE FechaPago > '2009-01-31'
EXCEPT
SELECT CodigoCliente
FROM pedidos
WHERE FechaPedido > '2009-03-31';
```

Los Codigos de Cliente retornados por la consulta son: 3, 15 y 19.

## Consultas multitabla mediante SUBCONSULTAS

Una **subconsulta** es una instrucción `SELECT` que se usa dentro de otra instrucción `SELECT`.

Una **subconsulta** será **correlacionada** si aparecen datos de la consulta, es decir, si no se puede ejecutar de forma independiente.

Existen varias situaciones donde usaremos subconsultas. A continuación veremos algunos ejemplos.

### Subconsultas en `WHERE` con operador de comparación

Para el siguiente ejemplo, primero buscamos la cantidad media que pagan los clientes.

```sql
SELECT AVG(Cantidad) AS CantidadMedia
FROM pagos;
```

### Ejemplo 1

**Subconsulta en `WHERE` con operador de comparación**

Mostrar los registros de pagos que tengan cantidades superiores a la media.

```sql
SELECT *
FROM pagos
WHERE Cantidad > (SELECT AVG(Cantidad) AS CantidadMedia FROM pagos);
```

## Subconsultas en `WHERE` con operador `IN`

El operador `IN` devuelve verdadero si el valor del campo de un registro está en el conjunto de valores devuelto por la subsonsulta.

### Ejemplo 2

**Subconsulta en `WHERE` con operador `IN`**

Mostrar la `Gama` de los productos de los que se haya pedido más de 30 unidades.

```sql
SELECT DISTINCT Gama
FROM productos
WHERE CodigoProducto IN (SELECT CodigoProducto FROM detallepedidos WHERE Cantidad > 30);
```

También puede usarse de forma negativa con `NOT IN`

## Subconsultas en `WHERE` con operador `EXISTS`

El operador `EXISTS` es verdadero si la subconsulta devuelve al menos un registro.

### Ejemplo 3

**Subconsulta en `WHERE` con operador `EXISTS`**

Obtener los datos de los empleados que tengan algún cliente asignado.

```sql
SELECT *
FROM empleados
WHERE EXISTS (SELECT * FROM Clientes WHERE CodigoEmpleadoRepVentas = empleados.CodigoEmpleado);
```

También puede usarse de forma negativa con **NOT EXISTS**

### Subconsultas en `FROM`

Podemos utilizar subconsultas como tablas y colocarlas en la cláusula `FROM`.

#### Ejemplo 4

**Subconsulta en `FROM`**
Aunque la siguiente consulta se puede obtener mediante una consulta de intersección típica, usaremos una subconsulta para probar su funcionamiento en `FROM` de forma sencilla. Las subconsultas de `FROM` deben tener un alias que signaremos con `AS`.
Mostrar los datos de los empleados que trabajen en oficinas de Madrid.

```
SELECT empleados.*
FROM empleados, (SELECT * FROM oficinas WHERE Ciudad = 'Madrid') AS OficinasMadrid
WHERE empleados.CodigoOficina = OficinasMadrid.CodigoOficina;
```

La consulta anterior sin usar subconsulta sería:

```
SELECT empleados.*
FROM empleados, oficinas
WHERE empleados.CodigoOficina = oficinas.CodigoOficina AND oficinas.Ciudad = 'Madrid';
```

# Transacciones y bloqueos

## Transacciones

Las bases de datos son fundamentales para la gestión de la información en la mayoría de las aplicaciones informáticas. **Es común que múltiples aplicaciones accedan a la base de datos de manera concurrente**, es decir, que varios procesos realicen operaciones sobre la misma base de datos simultáneamente. En este contexto, **es crucial garantizar la integridad de los datos y la consistencia de la información.**

#### Ejemplo

Pongamos como ejemplo una aplicación de reserva de plazas para clases en un gimnasio.
Si dos usuarios intentan reservar la misma plaza al mismo tiempo, el sistema debe garantizar que solo uno de ellos pueda reservar la plaza. Si no se toman medidas, es posible que ambos usuarios reserven la plaza, lo que posiblemente llevaría a un número de reservas superior al número de plazas disponibles.
En este contexto, las transacciones, bloqueos y concurrencia son fundamentales.

#### Definición

> Una transacción es una secuencia de operaciones que se tratan como una unidad de trabajo. Esto significa que, todas las operaciones se completan con éxito, o bien ninguna de ellas se aplica.

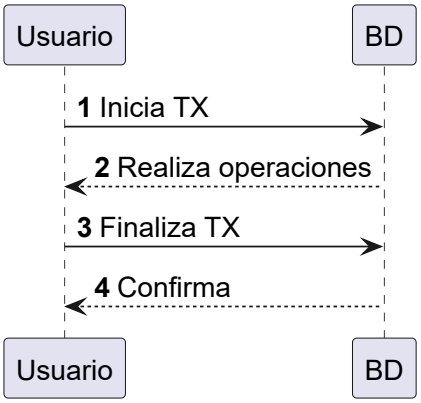
Imagina que estás haciendo una transferencia bancaria de una cuenta a cuenta a mi cuenta. Esta operación se divide en dos pasos:

- Restar la cantidad transferida del saldo de tu cuenta.
- Sumar la cantidad transferida al saldo de mi cuenta.

Estos dos pasos forman una transacción. Si ambos pasos se completan con éxito, la transferencia se realiza correctamente. Si uno de los pasos falla, la transacción se cancela y se revierten los cambios realizados.

Una transacción SQL es un conjunto de sentencias SQL que se ejecutan de forma indivisible (atómica), como si fueran una única
operación. El uso de las transacciones es esencial para mantener la integridad de los datos.
Los pasos a la hora de realizar una transacción se resumen en:

1. Iniciar la transacción
2. Realizar las operaciones
3. Finalizar la transacción, confirmando los cambios o deshaciéndolos.




## Transacciones en MariaDB

MariaDB sólo permite realizar transacciones con el motor de almacenamiento InnoDB.
Para realizar una transacción:

1. Indicar que vamos a realizar una transacción con `START TRANSACTION`, `BEGIN` o `BEGIN WORK`
2. Realizar las operaciones de manipulación de datos sobre la base de datos (insertar, actualizar o borrar filas)
3. Si las operaciones se han realizado correctamente y queremos que los cambios se apliquen de forma permanente sobre la base de
   datos usaremos la sentencia `COMMIT` . Sin embargo, si durante las operaciones ocurre algún error y no queremos aplicar los cambios
   realizados podemos deshacerlos con la sentencia `ROLLBACK` .
   En el siguiente enlace puedes encontrar más información sobre las transacciones en MariaDB:
   https://mariadb.com/kb/en/transactions/

### Autocommit

En MariaDB, por defecto, cada sentencia SQL se ejecuta en una transacción independiente. Esto quiere decir que, si ejecutamos una
sentencia SQL que modifica la base de datos, los cambios se aplicarán de forma permanente.
Este comportamiento se conoce como `AUTOCOMMIT` . Podemos consultar el valor actual de `AUTOCOMMIT` con la siguiente sentencia SQL:

```
SELECT @@AUTOCOMMIT;
```

Para desactivarla y poder trabajar con transacciones, hacemos:

```
SET AUTOCOMMIT = 0;
```

Si ejecutamos la sentencia anterior, tendremos una transacción abierta y los cambios sólo se aplicarán en la BD ejecutando la sentencia
`AUTOCOMMIT` ; .
Para volver a activar el `AUTOCOMMIT` :

```
SET AUTOCOMMIT = 1;
```

#### Ejemplo 1

Dadas las siguientes instrucciones

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;
CREATE TABLE cliente (
    id INT UNSIGNED PRIMARY KEY,
    nombre CHAR (20)
);

START TRANSACTION;
INSERT INTO cliente VALUES (1, 'Pepe');
COMMIT;
```

1. ¿Qué devolverá esta consulta?

```
SELECT * FROM cliente;

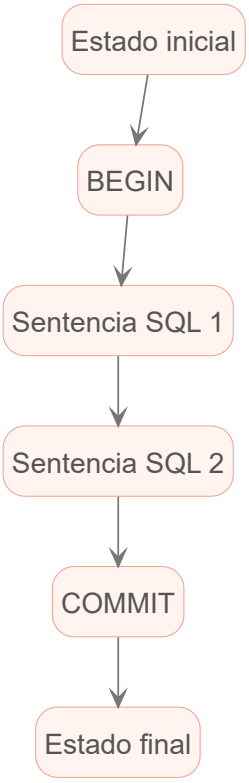
SET AUTOCOMMIT=0;
INSERT INTO cliente VALUES (2, 'Maria');
INSERT INTO cliente VALUES (20, 'Juan');
DELETE FROM cliente WHERE nombre = 'Pepe';
```

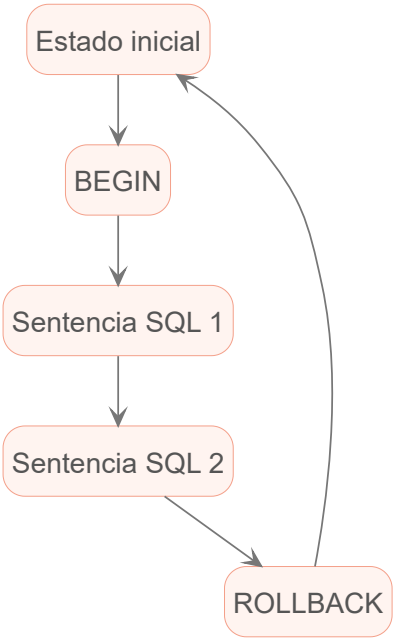
2. ¿Qué devolverá esta consulta?

```
SELECT * FROM cliente;
ROLLBACK;
```

3. ¿Qué devolverá esta consulta?

```
SELECT * FROM cliente;
```




### Ejemplo 2

Dadas las siguientes instrucciones

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;

CREATE TABLE cuentas (
    id INTEGER UNSIGNED PRIMARY KEY,
    saldo DECIMAL(11,2) CHECK (saldo >= 0)
);

INSERT INTO cuentas VALUES (1, 1000);
INSERT INTO cuentas VALUES (2, 2000);
INSERT INTO cuentas VALUES (3, 0);

-- Estado actual de las cuentas
SELECT * FROM cuentas;
```

1. Se realiza una transferencia entre dos cuentas, ¿qué devolverá la consulta?

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;

-- Consulta
SELECT * FROM cuentas;
```

2. Se realiza ahora esta operación , ¿qué devolverá la consulta?

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 9999;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;

-- Consulta
SELECT * FROM cuentas;
```

3. Se realiza ahora otra transacción, ¿qué devolverá la consulta?

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 3;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;

-- Consulta
SELECT * FROM cuentas;
```

## Transacciones concurrentes

En un entorno de bases de datos, la concurrencia se refiere a la capacidad del sistema para permitir que múltiples usuarios o
aplicaciones accedan y modifiquen los datos de la base de datos al mismo tiempo.
Sin los servicios de control de la concurrencia apropiados en un sistema gestor de bases de datos o con la falta de conocimiento sobre
cómo usar estos servicios de forma adecuada, el **contenido** en la base de datos o los **resultados** de nuestras consultas podrían
corromperse, y dejar de ser fiables.

## Pérdida de actualizaciones (Lost Updated)

Dos transacciones leen el mismo dato. Una transacción lo modifica y lo confirma. La segunda transacción, sin haber visto el cambio de la
primera, también modifica el dato y lo confirma, sobrescribiendo el cambio anterior.

### Ejemplo 3

Vamos a simular un caso de transacciones concurrentes que pueden llevar a una perdidad de actualizaciones.

Partimos de la siguiente base de datos

```
SET AUTOCOMMIT=1;
DROP DATABASE IF EXISTS evento;
CREATE DATABASE evento;
USE evento;

DROP TABLE IF EXISTS asientos;
CREATE TABLE asientos (
    asiento INT UNSIGNED NOT NULL, -- Asiento: número entero positivo
    estado ENUM ('libre', 'ocupado', 'no disponible') NOT NULL, -- Estado: solo puede tener uno de los tres valores i
    usuario INT UNSIGNED, -- Usuario: número entero positivo (puede ser nulo)
    PRIMARY KEY (asiento) -- Clave primaria para identificar unívocamente cada asiento
);

INSERT INTO asientos (asiento,estado) VALUES (1,'libre'), (2,'libre'), (3,'libre'), (4,'libre'), (5,'libre');
```

Verifica que tienes activado el AUTOCOMMIT en los scripts de ambos usuarios y realiza las siguientes operaciones:

Ejecuta estas sentencias en el orden indicado

| Usuario 1 | Usuario 2 |
|---|---|
| SELECT * FROM asientos WHERE estado='libre' | - |
| - | SELECT * FROM asientos WHERE |
| UPDATE asientos SET estado='ocupado', usuario=1 WHERE asiento=3 | - |
| - | UPDATE asientos SET estado='oc |

**Consecuencia**: La actualización de la primera transacción se pierde, lo que puede llevar a inconsistencias en los datos.

Si introducimos transacciones para evitar las condiciones de perdida de actualizaciones, aparecen otro tipo de problemas:

- Problema de lectura sucia.
- Problema de lectura no repetible.
- Problema de lectura fantasma.

## Problema de lectura sucia (Dirty Read)

Una transacción lee un dato modificado por otra transacción que aún no ha confirmado sus cambios. Si la transacción que ha modificado
el dato se revierte, la transacción que ha leído el dato modificado tendrá información incorrecta.

| **Usuario 1 (Transacción A)** | **Usuario 2 (Transacción B)** |
|---|---|
| ¿Cuál es el balance actual? | Sacaré 500€ |
| | (1) Leer cuenta x |
| | (2) balance = balance - 500 |
| | (3) Actualizar balance cuenta x |
| (4) Leer el balance de la cuenta x | |
| | ROLLBACK |

La anomalía de lectura sucia significa que la transacción acepta el riesgo de leer datos no fiables (no confirmados) que podrían cambiar o actualizarse con datos que podrían ser deshechos (ROLLBACK). Este tipo de transacción no debe hacer ninguna actualización en la base de datos ya que podría llevar a datos corruptos. De hecho, cualquier uso de datos sin confirmar es arriesgado y puede llevar a decisiones o acciones incorrectas.

## Problema lectura no repetible (Non-Repeatable Read)

Se produce cuando en una transacción se consulta el mismo dato dos veces, y la segunda vez encuentra que el valor del dato ha sido modificado por otra transacción.
Supongamos una aplicación web de reserva de asientos para un concierto de música. Hay dos usuarios que

inician la reserva del mismo asiento. El usuario 1 consulta el estado de los asientos y comprueba que el asiento 33 está libre. Mientras tanto, el usuario 2 reserva el
asiento 33, y cuando el usuario 1 intenta reservar el asiento 33, se encuentra con que ya no está disponible.

| **Usuario 1 (Transacción A)** | **Usuario 2 (Transacción B)** |
|---|---|
| SELECT estado FROM asientos WHERE num=33; | |
| | UPDATE asientos SET estado ='ocupado' WHERE num=33; |
| SELECT estado FROM asientos WHERE num=33; | |

## Problema de lectura fantasma (Phantom Read)

Se trata de una variación de lecturas no repetibles. Este error ocurre cuando una transacción ejecuta dos veces una consulta que devuelve un conjunto de filas y en la segunda ejecución de la consulta aparecen/desaparecen nuevas filas en el conjunto que no existían cuando se inició lo transacción.
Supongamos que el usuario 1 comprueba los asientos disponibles. A la vez, el administrador de la aplicación elimina el asiento 33 ya que se ha roto y no debe aparecer entre los asientos existentes. Cuando el usuario 1 vuelve a comprobar los asientos existentes, comprueba su estado y el asiento no aparece, cuando antes sí (de ahí que sea una lectura fantasma).

| **Usuario 1 (Transacción A)** | **Usuario 2 (Transacción B)** |
|---|---|
| SELECT num FROM asientos WHERE estado='libre'; | |
| | DELETE FROM asientos WHERE num =33; COMMIT; |
| SELECT num FROM asientos WHERE 'estado ='libre'; | |

## Propiedades ACID de la transacción ideal

Todo SGBD que se considera transaccional debe cumplir con las propiedades ACID:
Atomicidad: La transacción es indivisible. Esto quiere decir, que o se ejecutan todas la sentencias o no lo hará ninguna.

- **Consistencia**: Después de una transacción, la base de datos estará en un estado válido y consistente.
- **Aislamiento** (Isolation): Cada transacción está aislada del resto de transacciones y el acceso a los datos se hace de forma exclusiva. Si una transacción quiere acceder de forma concurrente a los datos que están siendo utilizados por otra transacción, no podrá hacerlo hasta que la primera haya terminado.
- **Durabilidad**: Los cambios que realiza una transacción sobre la base de datos son permanentes.

### Niveles de aislamiento

Para evitar que sucedan los problemas de acceso concurrente que hemos comentado en el punto anterior podemos establecer diferentes niveles de aislamiento que controlan el nivel de bloqueo durante el acceso a los datos. El estándar ANSI/ISO de SQL (SQL92) define cuatro niveles de aislamiento:

- **READ UNCOMMITTED** (lectura no confirmada/no acometida/no comprometida): Es el nivel de aislamiento más bajo. Permite que una transacción lea datos que han sido modificados por otra transacción pero que aún no han sido confirmados. Esto puede llevar a problemas de lectura sucia, lectura no repetible y lectura fantasma.
- **READ COMMITTED**: En este nivel de aislamiento, una transacción no puede leer datos que han sido modificados por otra transacción y que aún no han sido confirmados. Sin embargo, una transacción puede leer datos que han sido modificados y confirmados por otra transacción.
- **REPEATABLE READ**: En este nivel de aislamiento, una transacción no puede leer datos que han sido modificados por otra transacción y que aún no han sido confirmados. Además, una transacción no puede modificar datos que han sido leídos por otra transacción y que aún no han sido confirmados.
- **SERIALIZABLE**: Es el nivel de aislamiento más alto. En este nivel, una transacción no puede leer datos que han sido modificados por otra transacción y que aún no han sido confirmados. Además, una transacción no puede modificar datos que han sido leídos por otra transacción y que aún no han sido confirmados

## Bloqueos

Cuando un SGBD aísla las transacciones de usuario, utiliza los bloqueos para restringir el acceso de otros usuarios a dichos datos.
Los bloqueos pueden ser Compartidos o Exclusivos. Para entener la diferencia entre ambos bloqueos vamos a exponer algunos casos de uso.

#### Casos de uso de bloqueos compartidos

- **Generación de informes**. Al generar un informe que requiere leer datos de varias tablas, se puede usar un bloqueo compartido para garantizar que los datos no cambien durante la generación del informe, evitando así inconsistencias en los resultados.
- **Control de acceso concurrente**: En aplicaciones donde varios usuarios pueden acceder a los mismos datos, se pueden usar bloqueos compartidos para permitir la lectura concurrente de los datos, pero evitar modificaciones simultáneas que podrían generar inconsistencias.

#### Casos de uso de bloqueos exclusivos

- **Transferencia bancaria**. Al transferir dinero entre dos cuentas, se necesita un bloqueo exclusivo en ambas cuentas para evitar que otras transacciones modifiquen los saldos de forma simultánea. De lo contrario, se podrían producir inconsistencias en los saldos.
- **Actualización de inventario**. Al actualizar el inventario de una tienda, se necesita un bloqueo exclusivo en los productos afectados para evitar que otros usuarios realicen ventas o modificaciones en el inventario al mismo tiempo.

En resumen:

- **Bloqueos exclusivos**: Se usan cuando se necesita modificar datos y se quiere evitar que otras transacciones accedan a ellos mientras se realizan las modificaciones.
- **Bloqueos compartidos**: Se usan cuando se necesita leer datos y se quiere evitar que otras transacciones los modifiquen mientras se están leyendo, pero se permite que otras transacciones lean los mismos datos.

## Niveles de bloqueo

Los bloqueos se pueden realizar a diferentes niveles:

- **Nivel de fila**: Bloquea únicamente las filas específicas que una transacción necesita modificar, permitiendo que otras transacciones accedan a otras filas de la misma tabla.
- **Nivel de tabla**: Bloquea toda la tabla, impidiendo que otras transacciones la modifiquen mientras la transacción actual la esté utilizando.

Los bloqueos explicitos pueden utilizarse en cualquier nivel de aislamiento. Sin embargo, su uso es más común en los niveles de aislamiento más bajos, donde el control de concurrencia es menos estricto.
Los bloqueos quedán liberados cuando se realiza un `COMMIT` o un `ROLLBACK` de la transacción.

### Bloqueo exclusivo a nivel de fila

En MariaDB, la gestión explícita de bloqueos de filas se realiza principalmente a través de la cláusula `FOR UPDATE` en las sentencias `SELECT` . Esta cláusula permite a una transacción adquirir bloqueos exclusivos sobre las filas seleccionadas, asegurando que ninguna otra transacción pueda modificarlas hasta que la transacción actual libere los bloqueos.

#### Ejemplo 4

Vamos a simular que una transacción de un usuario 1 inicia el proceso de compra de un asiento para un evento. Para ello, inicialmente va a proceder a la lectura de los asientos libres, posteriormente y una vez que el usuario 1 ha elegido el asiento lo bloqueará de manera compartida hasta que finalice el proceso de compra, y así ningún otro usuario pueda comprar el mismo asiento.

Partimos de la siguiente base de datos

```
DROP TABLE IF EXISTS asientos;
    CREATE TABLE asientos (
    asiento INT UNSIGNED NOT NULL, -- Asiento: número entero positivo
    estado ENUM ('libre', 'ocupado', 'no disponible') NOT NULL, -- Estado: solo puede
    usuario INT UNSIGNED, -- Usuario: número entero positivo (puede ser nulo)
    PRIMARY KEY (asiento) -- Clave primaria para identificar unívocamente cada asient
);
INSERT INTO asientos (asiento,estado) VALUES
    (1,'libre'), (2,'libre'), (3,'libre'), (4,'libre'), (5,'libre');
```

Usuario 1 inicia el proceso de compra de un asiento

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;
-- En este punto el usuario 1 consulta todos los asientos libres
SELECT * FROM asientos WHERE estado='libre';
-- El usuario 1 elige el asiento 2 y lo bloquea para iniciar el proceso de compra
SELECT * FROM asientos WHERE estado='libre' AND asiento=2 FOR UPDATE;
```

Usuario 2 inicia también el proceso de compra de un asiento

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

-- El usuario2 puede consultar el estado de los asientos
SELECT * FROM asientos WHERE estado='libre';
-- Pero no puede modificar el asiento 2
SELECT * FROM asientos WHERE estado='libre' AND asiento=2 FOR UPDATE;
UPDATE asientos SET estado='ocupado', usuario='2' WHERE asiento=2;

-- Sin embargo si puedo modificar el asiento 3.
SELECT * FROM asientos WHERE estado='libre' AND asiento=3 FOR UPDATE;
UPDATE asientos SET estado='ocupado', usuario='2' WHERE asiento=3;
```

Si el usuario 1 confirma o deshace la transacción libera el bloqueo.

```
UPDATE asientos SET estado='ocupado', usuario='1' WHERE asiento=2;
COMMIT;
```

El usuario 2 finaliza

```
COMMIT;
```

La transacción del usuario 2 se bloqueará hasta que el usuario 1 libere el bloqueo con un `COMMIT` o `ROLLBACK`;

**Bloqueo compartido a nivel de fila**

Cuando un usuario ejecuta una consulta SELECT para obtener detalles de un producto específico, MariaDB puede otorgar un bloqueo compartido a nivel de fila sobre ese registro en particular. Esto permite que otros usuarios también lean la misma fila sin ningún
problema.

La forma más común de "configurar" un bloqueo compartido a nivel de fila es a través de la cláusula `LOCK IN SHARE MODE` en una sentencia `SELECT` .

**Ejemplo 5**

Vamos a simular que el gestor de una compañia de aviación quiere saber los asientos libres y ocupados de los vuelos de la compañia. Para ello, durante el tiempo que tarda en generar el informe bloqueará de manera compartida los asientos de los vuelos, y así ningún otro usuario puede modificar los asientos.

Partimos de la siguiente base de datos

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

-- En este punto el usuario 1 consulta todos los asientos libres
SELECT COUNT(*) FROM asientos WHERE estado='libre' LOCK IN SHARE MODE;
```

**Bloqueo compartido a nivel de tabla**

Los bloqueos explícitos a nivel de tabla permiten a los usuarios controlar manualmente el acceso a una tabla completa, impidiendo que otras transacciones realicen modificaciones no deseadas.
Esto es especialmente útil en situaciones donde se requiere una alta consistencia de los datos, como durante la ejecución de tareas complejas o la realización de modificaciones masivas.

**No es recomendable usar bloqueos a nivel de tabla a menos que sea absolutamente necesario**, ya que pueden causar cuellos de botella y reducir significativamente la concurrencia en la base de datos.

# Usuarios y privilegios

La gestión de usuarios es una tarea esencial en la gestión de entornos de bases de datos. Al asiginar cuentas de usuario únicas, puedes controlar el acceso a diferentes partes de la base de datos, asegurando que cada usuario tenga los privilegios apropiados. Este
proceso mejora la seguridad y ayuda a organizar el acceoso entre diferentes aplicaciones o usuarios. Se puede administrar las cuentas de usuario con la declaración `CREATE USER` y `DROP USER` , y asignar privilegios con `GRANT` y `REVOKE` .

MariaDB almacena **información del usuario** en sus tablas de sistema (base de datos mysql) en la tabla **mysql.global_priv**.

### Ejemplo 5

Podemos acceder a la información de los usuarios con las siguientes consultas:

```
USE mysql;
SELECT user, host FROM global_priv;
```

**Usuarios**

| user | host |
|---|---|
| PUBLIC | |
| root | 127.0.0.1 |
| root | ::1 |
| | desktop-shptuam |
| root | desktop-shptuam |
| | localhost |
| mariadb.sys | localhost |
| root | localhost |

La columna *Host* indica desde qué equipo puede conectarse un usuario al servidor MariaDB. En este caso, encontramos:

- *root*: aparece cuatro veces porque tiene permitido conectarse desde distintos orígenes: `localhost`, `127.0.0.1` (dirección IPv4 local), `::1` (dirección IPv6 local) y `desktop-shptuam` (nombre del equipo).
- *mariadb.sys*: es una cuenta interna de MariaDB, bloqueada para el inicio de sesión.
- *Usuarios anónimos*: son los registros que tienen el campo User vacío. Están asociados a `localhost` y `desktop-shptuam`.
- *PUBLIC*: es un rol especial de MariaDB que representa los privilegios públicos.

En MariaDB, cada cuenta se identifica mediante la combinación de User y Host, por lo que un mismo usuario puede aparecer varias veces.

El símbolo `%` funciona como comodín en la columna Host y permite indicar cualquier origen de conexión, aunque en este ejemplo no aparece ninguna cuenta que lo utilice.

## Creación de usuarios SQL

Para crear un nuevo usuario se realiza con la sentencia `CREATE USER`, mediante la sintaxis:

```
CREATE USER nombre_usuario IDENTIFIED BY 'contraseña';
```

Al indicar la contraseña, esta se encriptará con un *hash*. El hash es irreversible, lo que significa que no se puede recuperar la contraseña original a partir de él.

Además, en MariaDB, podemos especificar el host desde donde se permite la conexión:

```
-- En esta sentencia estamos indicando que el usuario solo se puede conectar desde la propia máquina donde está instalado el
CREATE USER 'nombre_usuario'@'localhost' IDENTIFIED BY 'contraseña';
```

Si necesitamos **cambiar la contraseña o actualizar las propiedades** de un usuario existente, utilizaremos la instrucción `ALTER USER`:

```
ALTER USER nombre_usuario IDENTIFIED BY 'nueva_contraseña';
```

Finalmente, para eliminar usuarios utilizaremos la sentencia `DROP USER`:

```
DROP USER nombre_usuario;
```

## Asignación de privilegios

Al administrar usuarios, es crítico asignar correctamente privilegios basados en su rol. Los privilegios determinan qué acciones pueden realizar los usuarios, como consultar datos o modificar bases de datos. Crear una cuenta separada para cada tarea asegura una estructura clara en su base de datos. Con acceso administrativo, puede crear, modificar y eliminar las cuentas de usuario según sea necesario.

La instrucción `GRANT` permite asignar permisos específicos a usuarios o roles, como la capacidad de consultar (`SELECT`), insertar (`INSERT`), actualizar (`UPDATE`) o eliminar (`DELETE`) datos en tablas o vistas. También se pueden otorgar privilegios sobre procedimientos almacenados o esquemas completos.

La sintaxis de GRANT es la siguiente:

```
GRANT privilegio1, privilegio2, ... ON nombre_base_datos.nombre_tabla TO 'nombre_usuario'@'host';
```

Los privilegios más comunes (los cuales podemos consultar con el comando `SHOW PRIVILEGES`) son:

- `USAGE`: para usar un objeto específico de la base de datos.
- `SELECT`: para acceder a tablas o vistas.
- `INSERT`: Si se especifica el nombre_columna, se otorga permiso para insertar en la columna especificada. Si se omite entonces se permite insertar valores en todas las columnas.
- `UPDATE`: Lo mismo que INSERT para modificar.
- `DELETE`: para eliminar registros de una tabla o vista.
- `WITH GRANT OPTION`: permite dar permisos a otro usuario.
- `ALL PRIVILEGES`: otorga todos los permisos menos GRANT OPTION

### Ejemplo 6

Podemos acceder a la información de los usuarios con las siguientes consultas:

```
CREATE DATABASE BalmisBD;
GRANT ALL PRIVILEGES ON BalmisBD.* TO 'uBalmis';
```

Con los comandos anteriores, hemos creado una base de datos llamada BalmisBD y hemos otorgado todos los permisos al usuario uBalmis sobre BalmisBD.
Después de los comandos anteriores habrá que ejecutar el siguiente comando para refrescar todos los privilegios a los usuarios:

```
FLUSH PRIVILEGES;
```

### Ejemplo 7

En este ejemplo vamos a crear la base de datos prestashop y el usuario `user@localhost`, y le vamos a asignar al usuario los permisos de `SELECT`, `INSERT`, `UPDATE` y `DELETE`.

```
-- Creamos la base de datos
CREATE DATABASE prestashop;

-- Creamos el usuario y le asignamos los permisos
CREATE USER user@localhost IDENTIFIED BY 'passwd';
GRANT SELECT, INSERT, UPDATE, DELETE ON prestashop.* TO user@localhost;

-- Refrescamos los privilegios si queremos que se apliquen los cambios inmediatamente
FLUSH PRIVILEGES;
```

### Ejemplo 8

En este ejemplo vamos a utilizar la opción `WITH GRAN OPTION` para que el usuario `user@localhost` pueda asignar permisos a otros usuarios.

```
-- Creamos la base de datos
CREATE DATABASE prestashop;

-- Creamos el usuario y le asignamos los permisos
DROP USER IF EXISTS user@localhost;
CREATE USER user@localhost IDENTIFIED BY 'passwd';
GRANT SELECT, INSERT, UPDATE, DELETE ON prestashop.*
   TO user@localhost WITH GRANT OPTION;

-- Refrescamos los privilegios si queremos que se apliquen los cambios inmediatamente
FLUSH PRIVILEGES;
```

Como el usuario `'user'@'localhost'` tiene la opción `WITH GRANT OPTION` y tiene todos los permisos (`ALL PRIVILEGES`) sobre todas las tablas de base de datos prestashop , podrá asignar asignar estos mismos permisos a otros usuarios sobre las mismas tablas de la base de datos `prestashop`.

### Ejemplo 9

En el siguiente ejemplo el usuario `user@localhost` sólo podrá asignar a otros usuarios el permiso de `SELECT` sobre la tabla `customer` de la base de datos `prestashop` .

```sql
-- Eliminamos el usuario si ya existe
DROP USER IF EXISTS 'user'@'localhost';

-- Creamos el usuario
CREATE USER 'user'@'localhost' IDENTIFIED BY 'passwd';

-- Concedemos el permiso SELECT sobre la tabla customery permitimos que pueda otorgarlo a otros usuarios
GRANT SELECT ON prestashop.customer TO 'user'@'localhost' WITH GRANT OPTION;
```

Por último, para saber los permisos del usuario actual, podemos ejecutar `SHOW GRANTS FOR 'nombre_usuario'[@host]`. Si queremos saber los permisos del usuario actual, podemos ejecutar:

```sql
SHOW GRANTS FOR CURRENT_USER;
```

## Eliminar permisos

## Creación de roles

# Optimización de consultas